Contact Center & Case Management Platform

Architectural Blueprint & Implementation Roadmap

MVP Edition • Open-Source Stack • Production-Ready Design

Version 1.0 | June 2025 | Senior Full-Stack Architecture Division

© 2025 CCMP Architecture Team

1. Executive Overview

This document presents a production-ready architectural blueprint for the Contact Center & Case Management Platform (CCMP). It covers system design, technology choices, data models, integration patterns, and a phased implementation roadmap — all using a 100% open-source stack optimized for a "start small, scale fast" deployment philosophy.

The architecture balances three primary concerns:

- Operational velocity — agents, supervisors, and QA analysts can work efficiently from day one.
- Engineering pragmatism — the stack is learnable, well-documented, and avoids vendor lock-in.
- Scalability headroom — containerized services can be scaled horizontally without re-architecture.

> 💡 This blueprint targets an experienced full-stack team of 3–5 engineers. A disciplined 4-phase delivery plan is provided to ensure incremental, testable value delivery.

2. Recommended Tech Stack

Every library and tool below is open-source, free-tier available, and actively maintained with strong community support.

Layer	Technology	Version	Rationale
Frontend	Next.js 14	14.x (App Router)	SSR + RSC for SEO-safe pages; API routes for BFF pattern
	Tailwind CSS + shadcn/ui	3.x + latest	Accessible components out of the box; zero-runtime CSS
	Zustand + React Query	4.x + 5.x	Lightweight state + server-state sync with caching
API Gateway	Node.js + Express	20 LTS + 4.x	schema validation built-in
	tRPC	11.x	End-to-end type safety between Next.js and API layer
Services	Python + FastAPI	3.12 + 0.115	Routing engine, SLA jobs, ML scoring — async-native
Primary DB	PostgreSQL 16	16.x	JSONB for flexible case metadata; pg_partman for scaling
	Prisma ORM	5.x	Type-safe queries; auto-migrations; excellent DX
Cache / RT	Redis 7 (Valkey)	7.x	Pub/sub for real-time events; session cache; rate limiting

© 2025 CCMP Architecture Team

Layer	Technology	Version	Rationale
Search	Meilisearch	1.x	Full-text case/audit search; easier ops than Elasticsearch
Real-time	Socket.io	4.x	WebSocket with Redis adapter for multi-node broadcasting
Job Queue	BullMQ	5.x	Redis-backed; retries, priorities, cron scheduling
Auth	Keycloak	24.x	Enterprise RBAC, SSO, OAuth2/OIDC, MFA, audit logs
File Storage	MinIO	latest	S3-compatible; self-hosted; AES-256 server-side encryption
Telephony	FreeSWITCH + WebRTC	1.10.x	Open-source PBX; browser-based calling via SIP.js
Infra	Docker + Docker Compose	26.x + 2.x	Local dev parity; production-ready single-host MVP
	Traefik	3.x	Reverse proxy + auto TLS; Docker-native service discovery
Observability	Grafana + Prometheus + Loki	latest	Metrics, logs, dashboards — full open-source observability stack
	OpenTelemetry	1.x	Distributed tracing across all services
Testing	Vitest + Playwright + k6	1.x + 1.x + 0.x	Unit / E2E / load testing — complete coverage strategy

> 💡 Start with Docker Compose for the MVP. When traffic justifies it, the same services migrate to Kubernetes without code changes — only deployment manifests change.

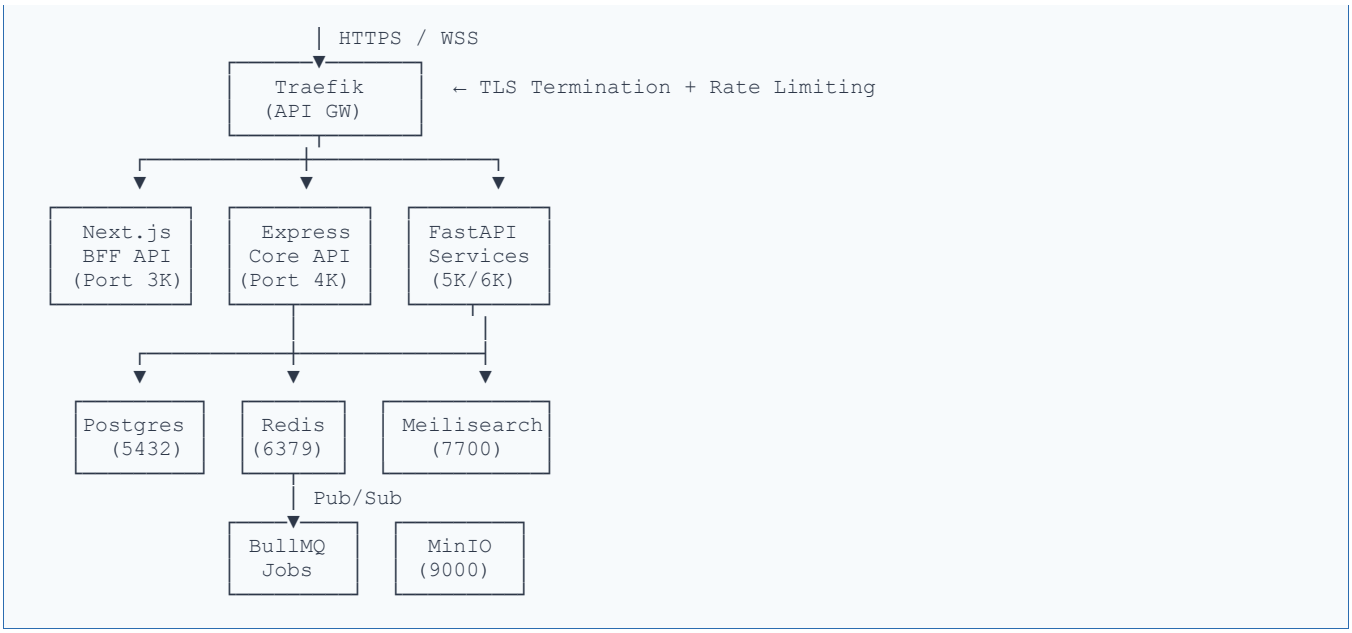
3. System Architecture

3.1 High-Level Architecture

The CCMP follows a modular monolith-first approach — a single deployable API with cleanly separated domain modules that can be extracted into microservices when needed. This avoids premature complexity while preserving the option to scale.

```
── ARCHITECTURE OVERVIEW ──

┌──────────────────────── CLIENT TIER ────────────────────────┐
│ Next.js 14 (Agent Desktop │ Supervisor │ QA │ Admin)
│ WebSockets (Socket.io)  │  REST/tRPC │  WebRTC SIP
└──────────────────────────────────────────────────────────────
                          │
```



3.2 Core Domain Services

Service	Responsibility	Key Patterns
Case Service	CRUD for case lifecycle; state machine transitions; attachment management	Command/Query separation; Event sourcing for audit trail
Routing Engine	Rule evaluation → queue assignment; agent skill matching; workload balancing	Strategy pattern; rule DSL; Redis-backed queue state
SLA Service	Countdown timers per case; breach detection; escalation chain triggers	BullMQ delayed jobs; priority queues; Redis TTL
Notification Service	Agent alerts; CSAT triggers; daily summaries; webhook delivery	Fan-out pub/sub; template engine; retry with backoff
Recording Service	Upload to MinIO; AES-256 encryption; role-gated playback URLs	Presigned URLs; server-side encryption; access log
Audit Service	Immutable event log; compliance export; access history	Append-only table; write-ahead log tailing; WORM semantics
Auth / RBAC	JWT validation; permission enforcement; session management	Keycloak OIDC; middleware-level policy; attribute-based access
Search Service	Full-text case search; audit log search; knowledge base index	Meilisearch index sync; async indexing on write
Telephony Adapter	FreeSWITCH event bridge; call metadata ingestion; CDR storage	ESL socket listener; adapter pattern; event normalization

© 2025 CCMP Architecture Team

4. Core Data Model

4.1 Database Schema Design

PostgreSQL 16 is the single source of truth. The schema uses standard relational tables for structured data and JSONB columns for extensible metadata (e.g., channel-specific case details, routing rule outputs).

Primary Tables

Table	Key Columns	Index Strategy	Notes
users	id, role, keycloak_id, team_id	idx on role, team_id	Synced from Keycloak via webhook on user events
cases	id, status, priority, channel, customer_id, assigned_to, sla_due_at, metadata JSONB	Composite: (status, priority, sla_due_at); GIN on metadata	Partitioned by created_at month via pg_partman at scale
case_events	id, case_id, event_type, actor_id, payload JSONB, created_at	idx on case_id; created_at; Partial on event_type	Immutable append-only. Powers audit trail and case timeline
sla_policies	id, priority, channel, first_response_sec, resolution_sec	idx on (priority, channel)	Configurable per priority/channel combination
routing_rules	id, conditions JSONB, actions JSONB, priority_order, active	GIN on conditions; idx on priority_order	Rules evaluated in priority order by routing engine
queues	id, name, skills_required[], language, max_capacity	idx on name	Agent-to-queue assignment tracked in Redis for low-latency
recordings	id, case_id, minio_key, duration_sec, encrypted, playback_roles[]	idx on case_id	Presigned URL generated on access; TTL 15 min
audit_logs	id, actor_id, resource_type, resource_id, action, ip_addr, session_id, ts	Composite: (actor_id, ts); (resource_id, resource_type)	Never deleted; exported for compliance on demand
qa_reviews	id, case_id, reviewer_id, score,	idx on case_id, reviewer_id	QA score feeds agent performance reports

Table	Key Columns	Index Strategy	Notes
	flags[], coaching_notes, created_at		

4.2 Case State Machine

Cases flow through a well-defined state machine. Invalid transitions are rejected at the service layer with appropriate error codes.

```
NEW ──▶ ASSIGNED ──▶ IN_PROGRESS ──▶ WAITING_ON_CUSTOMER
                        │                    │
                        ▼                    ▼
                     ESCALATED ◀──────────────┘
                        │
                        ▼
                     RESOLVED ──▶ CLOSED
                                    │
                                    ▼
                                 REOPENED ──▶ ASSIGNED
```

5. Real-Time Architecture

5.1 WebSocket Event Bus

Socket.io with the Redis adapter enables horizontal scaling across multiple Node.js instances. All real-time events are namespaced and room-scoped for efficient fan-out.

Event	Room / Namespace	Consumers
case:assigned	/cases — team:{teamId}	Agent desktop — shows new case in queue
case:status_changed	/cases — case:{caseId}	All viewers of that case; supervisor dashboard
queue:updated	/queues — queue:{queueId}	Supervisor queue monitor; routing engine
agent:presence_changed	/agents — team:{teamId}	Supervisor workload view; routing engine availability
sla:warning	/sla — agent:{agentId}	Agent desktop SLA badge; supervisor heatmap
sla:breached	/sla — team:{teamId}	Auto-escalation trigger; operations manager alert
call:connected	/telephony — agent:{agentId}	Agent desktop call controls; recording service

5.2 Background Jobs (BullMQ)

Job	Schedule	Description
sla_countdown	Per-case delayed job	Created on case assignment; cancels on resolution; triggers sla:warning at 80% elapsed
sla_breach_check	Every 60 seconds	Scans for cases past sla_due_at; fires escalation chain
case_aging_escalate	Every 4 hours	Escalates WAITING_ON_CUSTOMER cases older than 24h
csat_survey_trigger	15 min after CLOSED	Sends satisfaction survey via email/SMS integration
daily_summary_email	Daily 07:00 UTC	Supervisor / manager digest of prior day KPIs
recording_retention	Daily 02:00 UTC	Purges MinIO recordings past retention policy (default 90d)
search_index_sync	Every 5 minutes	Ensures Meilisearch is in sync with recent case writes

6. Security Architecture

6.1 Defence-in-Depth Layers

Layer	Control	Implementation
Transport	TLS 1.3 everywhere	Traefik auto-provisions Let's Encrypt certs; HSTS headers enforced
Auth	OAuth2 / OIDC via Keycloak	Short-lived JWTs (15 min); refresh token rotation; MFA enforced for Compliance/Admin
AuthZ	RBAC + ABAC	Middleware validates role on every route; resource ownership checked at query level
Data at Rest	AES-256 encryption	PII fields encrypted via pgcrypto; call recordings AES-256 in MinIO
PII Masking	Field-level tokenization	Credit card numbers masked; PII redacted in logs before shipping to Loki
Rate Limiting	Redis sliding window	Traefik plugin: 100 req/min per IP; stricter on auth endpoints
Audit	Immutable event log	Every case access, state change, and recording play logged to audit_logs; append-only
Session	Timeout + rotation	30-min inactivity timeout; session invalidated on role change

© 2025 CCMP Architecture Team

Layer	Control	Implementation
Recordings	Role-gated presigned URLs	Only QA/Supervisor can generate playback URL; TTL 15 min; access logged
Compliance Export	Signed PDF bundles	Audit export signed with server private key; tamper-evident

> ⚠️ If handling live payment data over voice, engage a PCI-DSS QSA before go-live. FreeSWITCH supports DTMF pause (pause recording during card capture) — implement this before MVP launch.

7. Phased Implementation Roadmap

The roadmap is structured in four phases. Each phase delivers testable, deployable value. Later phases build on earlier infrastructure without requiring re-work.

Phase 1 — Foundation (Weeks 1–6)

Goal: A working, secured skeleton. Agents can log in and create cases manually.

Week	Deliverables	Success Criteria
1–2	Docker Compose infra: Postgres, Redis, Keycloak, MinIO, Traefik, Meilisearch. Prisma schema with all core tables. Express API skeleton with health checks.	All services healthy; DB migrations run cleanly
2–3	Keycloak RBAC setup: all 7 roles configured. JWT middleware in Express. Next.js app with login flow, protected routes, role-aware navigation.	Each role sees only permitted nav items
3–4	Case CRUD: create, read, update, state transitions. Case event sourcing (append-only events). Email and web form intake adapters.	Case lifecycle from NEW → CLOSED works end-to-end
4–5	Basic routing engine v1: round-robin + skill matching. Queue management. Agent presence (online/busy/offline) via Redis.	New case auto-assigned to available agent
5–6	Socket.io integration: real-time case updates. Agent desktop with case grid, filters, notes, status update. Basic supervisor view.	Agent sees assignment in real-time without refresh

Phase 2 — Core Operations (Weeks 7–12)

Goal: Full SLA tracking, telephony integration, and supervisor tooling.

Week	Deliverables	Success Criteria
7–8	SLA engine: BullMQ countdown jobs per case, warning at 80%, breach detection, escalation chain. SLA heatmap on supervisor dashboard.	SLA breach fires escalation within 60 seconds
8–9	FreeSWITCH integration: ESL event listener, call metadata to case, CDR storage. WebRTC softphone in agent desktop via SIP.js.	Inbound call creates case; agent answers in browser
9–10	Recording service: FreeSWITCH records to MinIO, AES-256 encryption, role-gated presigned URL playback, access log.	Only QA/Supervisor can play recordings; access logged
10–11	Supervisor dashboard: live queue monitor, agent workload, SLA heatmap, force escalate, reassign, SLA override actions.	Supervisor can reassign a case in under 3 clicks
11–12	Meilisearch indexing for cases and audit events. Full-text search in agent desktop and compliance UI. Chat intake adapter (WebSocket).	Search returns results in under 200ms

Phase 3 — QA, Compliance & Analytics (Weeks 13–18)

Goal: QA workflow, compliance tooling, and operational reporting.

Week	Deliverables	Success Criteria
13–14	QA module: review queue, call playback, scoring rubric, flag compliance violations, coaching notes. QA score stored to qa_reviews.	QA analyst completes full review without leaving UI
14–15	Compliance module: immutable audit log viewer, access history, signed PDF export. Role restricted to Compliance Officer.	Compliance export is tamper-evident and auditable
15–16	Reporting: AHT, SLA breach rate, FCR, agent scores, queue backlog, CSAT. Recharts visualisations. Prometheus metrics ingestion.	Operations Manager can export weekly PDF report
16–17	CSAT survey trigger (BullMQ delayed job after close), survey UI, results stored and aggregated in reports.	CSAT data appears in reports within 24 hours of case close
17–18	Grafana dashboards for infra metrics; Loki log aggregation with PII redaction; OpenTelemetry tracing across all services.	P95 latency visible in Grafana; no PII in Loki

Phase 4 — Scale & Harden (Weeks 19–24)

Goal: Production hardening, performance validation, and scale-readiness.

© 2025 CCMP Architecture Team

Week	Deliverables	Success Criteria
19–20	k6 load tests: 500 concurrent agents, 10k cases/hour. Postgres query tuning, connection pooling via PgBouncer. Redis cluster mode.	P95 API response < 200ms under load
20–21	pg_partman table partitioning on cases and audit_logs. Prisma migration scripts for zero-downtime deploys. DB read replica for reports.	Reports query read replica; no contention with writes
21–22	Admin module: routing rule builder UI, SLA policy editor, user/team management, system config. Full audit trail for admin actions.	Admin can create routing rules without code change
22–23	Security audit: OWASP ZAP scan, dependency audit, Keycloak hardening, penetration test checklist. Rate limiting tuning.	Zero critical vulnerabilities in OWASP report
23–24	DR runbook: backup/restore procedures for Postgres and MinIO. Chaos testing. Go-live readiness checklist. Documentation sprint.	Full restore from backup completes in < 2 hours

8. API Design Conventions

8.1 RESTful Endpoints (Core)

Method	Endpoint	Roles	Description
POST	/api/v1/cases	Agent+	Create case from any channel; auto-triggers routing engine
GET	/api/v1/cases	Agent+	Paginated case list; scope filtered by role automatically
GET	/api/v1/cases/:id	Agent+	Case detail with events timeline
PATCH	/api/v1/cases/:id/status	Agent+	State transition; validates allowed transitions
POST	/api/v1/cases/:id/notes	Agent+	Add note with optional attachment (MinIO pre-sign)
POST	/api/v1/cases/:id/escalate	Agent+	Trigger escalation chain; creates audit event
GET	/api/v1/queues	Supervisor+	Live queue state with agent counts from Redis
POST	/api/v1/cases/:id/assign	Supervisor+	Force-assign to specific agent
GET	/api/v1/recordings/:id/url	QA, Supervisor	Generate presigned MinIO URL; 15-min TTL

© 2025 CCMP Architecture Team

Method	Endpoint	Roles	Description
POST	/api/v1/qa/reviews	QA Analyst	Submit QA review with score and flags
GET	/api/v1/reports/:type	Manager+	Generate report; streams CSV or PDF
GET	/api/v1/audit/logs	Compliance	Paginated audit log; filterable by actor/resource/date
POST	/api/v1/audit/export	Compliance	Signed PDF export of audit trail for date range

💡 Versioning strategy: prefix all routes with /api/v1/. When breaking changes are needed, deploy /api/v2/ alongside v1 and deprecate with a sunset header. Never break existing consumers.

9. Intelligent Routing Engine

9.1 Rule Evaluation Pipeline

The routing engine runs as a FastAPI microservice. It evaluates rules in priority order and returns an assignment decision. Rules are stored in Postgres and hot-reloaded into Redis on change.

Step	Stage	Logic
1	**Ingest Case Context**	Extract: channel, language, customer_tier, issue_type, priority from case metadata
2	**Load Active Rules**	Fetch from Redis (hot cache) sorted by priority_order ASC; fallback to Postgres on miss
3	**Evaluate Conditions**	Each rule's JSONB conditions evaluated against case context using Python rule interpreter
4	**Select Matching Rule**	First matching rule wins; if no match, fall through to DEFAULT_ROUND_ROBIN rule
5	**Resolve Target Queue**	Rule action specifies queue_id or agent_id; validate queue exists and is active
6	**Check Agent Availability**	Query Redis agent presence: prefer ONLINE agents with lowest workload in target queue
7	**SLA-Aware Override**	If case priority=HIGH and all agents busy: assign to least-loaded Senior Agent regardless of queue
8	**Emit Assignment**	Write assignment to Postgres; publish case:assigned event to Redis pub/sub; update queue counters

10. Architecture Tips & Tricks

10.1 Engineering Excellence Patterns

Pattern	How to Apply in CCMP
Optimistic Locking	Add version column to cases table. API rejects PATCH if version in request != DB version. Prevents concurrent agent edits corrupting state.
Idempotency Keys	All POST endpoints accept X-Idempotency-Key header. Store key+response in Redis for 24h. Safe to retry without duplicate case creation.
Dead Letter Queue	BullMQ failed jobs move to DLQ after 3 retries. DLQ monitored in Grafana. Failed SLA jobs alert on-call immediately.
Feature Flags	Use simple Postgres table for feature flags. New features (e.g., AI routing) toggled per team without deployment. Roll out incrementally.
Read/Write Separation	Reporting queries hit Postgres read replica. Routing engine reads from Redis. Writes only go to primary. Prevents analytics queries degrading ops.
Event Sourcing Lite	case_events is the source of truth for case history. Current state in cases table is a materialized projection, rebuilable from events. Audit = free.
Circuit Breaker	Wrap FreeSWITCH and email provider calls with opossum (Node.js circuit breaker). Telephony failure should not cascade to case creation.
Zero-Downtime Deploy	Use rolling restarts in Docker Compose. Database migrations via Prisma with backward-compatible changes (add columns, never rename in prod).
Soft Deletes	Never hard-delete cases or users. Add deleted_at timestamp. Filter in all queries. Required for compliance; prevents referential integrity issues.
PII in Logs	Implement a log scrubber middleware that redacts email, phone, credit card patterns before logs ship to Loki. Use structured logging (Pino/structlog).

10.2 Common Pitfalls to Avoid

- Do not use DB-level timestamps for SLA calculation — use BullMQ delayed jobs instead. DB timestamps drift; job queues are reliable.
- Do not fan-out WebSocket events directly from the API handler — always go through Redis pub/sub so multi-instance deploys work correctly.
- Do not store session state in Node.js memory — use Redis. Any pod restart will invalidate in-memory sessions and log agents out unexpectedly.
- Do not index every JSONB field eagerly — profile first. GIN indexes on large JSONB columns are expensive on writes. Add targeted functional indexes instead.
- Do not skip connection pooling — PgBouncer in transaction mode is essential at scale. Direct connections to Postgres exhaust quickly under 500+ concurrent agents.
- Do not hardcode routing rules in code — the Postgres + Redis rule store means operations teams can adjust routing without a deployment.

10.3 Performance Benchmarks to Target

© 2025 CCMP Architecture Team

Metric	Target	How to Achieve
Case list API (p95)	**< 150ms**	Composite index on (status, priority, sla_due_at); paginate with cursor not OFFSET
Real-time event delivery	**< 100ms**	Redis pub/sub + Socket.io; avoid DB round-trip in event path
Routing decision	**< 50ms**	Rules hot-cached in Redis; agent availability from Redis hash; no DB query on hot path
Search results	**< 200ms**	Meilisearch primary filter on indexed fields; async index sync acceptable
Recording playback start	**< 2s**	Presigned URL generation < 50ms; MinIO streaming with range request support
Concurrent agents supported	**500 (MVP)**	PgBouncer + Redis cluster; horizontal Socket.io nodes with Redis adapter

11. Docker Compose Service Map

The following services should be defined in docker-compose.yml. All services share a ccmp-network bridge network. Health checks gate startup order.

Service	Port	Depends On	Notes
traefik	80, 443	—	Reverse proxy; auto-discovers services via Docker labels
postgres	5432	—	Primary DB; pg_partman extension; persistent volume
pgbouncer	6432	postgres	Connection pooler; transaction mode; pool size = 20 per service
redis	6379	—	Cache + pub/sub + BullMQ backend; persistent AOF
keycloak	8080	postgres	Auth server; realm config imported via JSON on first boot
minio	9000, 9001	—	Object storage; console on 9001; AES-256 encryption at rest
meilisearch	7700	—	Full-text search; master key from env secret
api (Express)	4000	postgres, redis, keycloak	Core API; multiple replicas behind Traefik for scale

© 2025 CCMP Architecture Team

Service	Port	Depends On	Notes
routing-service (FastAPI)	5000	postgres, redis	Routing engine; stateless; scalable independently
worker (BullMQ)	—	postgres, redis	Background job processor; runs SLA, notifications, reports
nextjs	3000	api	Frontend; server-side rendering; connects to api via internal network
freeswitch	5060, 7443	—	SIP server; ESL on 8021; WebRTC gateway for browser calling
prometheus	9090	—	Metrics scraping from all services via /metrics endpoint
grafana	3001	prometheus	Dashboards + Loki log explorer; Keycloak SSO login
loki	3100	—	Log aggregation; agents ship via Promtail with PII scrubbing

12. Recommended Team Structure

Role	Count	Primary Ownership
Tech Lead / Architect	1	System design, cross-cutting concerns, code review, ADRs
Full-Stack Engineer (Node/Next)	2	API layer, frontend, Socket.io, BullMQ jobs
Backend Engineer (Python)	1	Routing engine, FastAPI services, SLA logic
DevOps / Platform Engineer	1	Docker Compose, Traefik, Grafana, CI/CD pipeline, security hardening

> 💡 Maintain an Architecture Decision Record (ADR) file for every major technical choice. Tools like adr-tools make this trivial. ADRs prevent re-litigating the same decisions 6 months later.

Document Metadata

Document Version	1.0
Classification	Confidential — Internal Architecture
Stack Philosophy	100% Open-Source / Free-Tier

© 2025 CCMP Architecture Team

MVP Timeline	24 Weeks (6 Sprints of 4 Weeks)
Target Team Size	5 Engineers
Min. Infrastructure	1× VPS with 8 vCPU / 32GB RAM (MVP); 3-node cluster (production)
Primary Author	Senior Full-Stack Architecture Division

© 2025 CCMP Architecture Team